

Vietnam National University, Hanoi

University of Engineering and Technology

Xiangqi Game Engine

Môn học: INT3401E – Artificial Intelligence

Thành viên: Nguyễn Phan Việt Hưng
Nguyễn Minh Khiêm
Nguyễn Việt Dũng
Đỗ Đức Long

Repository: github.com/Oxigo-1212/INT3401E-2-project

Tóm tắt nội dung

Báo cáo này trình bày thiết kế và cài đặt các thuật toán Trí tuệ nhân tạo cho engine game Cờ Tướng (Xiangqi) trong khuôn khổ bài tập lớn môn INT3401E. Engine tích hợp một pipeline tìm kiếm gồm: thuật toán Negamax với cắt tỉa Alpha-Beta, Null-Move Pruning, Iterative Deepening kết hợp Aspiration Windows, và Quiescence Search. Thứ tự nước đi được tối ưu qua pipeline bốn tầng (Hash Move, MVV-LVA, Killer Heuristic, History Heuristic), kết hợp Bảng hoán vị (Transposition Table) với Zobrist Hashing để tránh tái đánh giá trạng thái trùng lặp. Hàm đánh giá tuyến tính năm thành phần (vật chất, vị trí PST, an toàn Tướng, cơ động, Xe mở cột) được nội suy theo pha trò chơi (tapered evaluation). Engine còn tích hợp Sổ khai cuộc (Opening Book) mã hóa các biến thể chuẩn. Engine hỗ trợ giao thức UCCI (Universal Chinese Chess Interface) nhằm kết nối dữ liệu với giao diện người dùng (GUI), hướng tới việc chuẩn hóa đầu ra để tham gia thi đấu và đo lường sức mạnh với các engine khác

Mục lục

1 Giới thiệu	3
2 Biểu diễn bàn cờ và sinh nước đi	3
2.1 Mô hình bàn cờ	3
2.2 Mã hóa quân cờ	4
2.3 Sinh nước đi hợp lệ	5
3 Thuật toán tìm kiếm	5
3.1 Từ Minimax đến Negamax	5
3.2 Cắt tỉa Alpha-Beta	6
3.3 Điểm số kết thúc ván	6
4 Cắt tỉa Null Move	6
5 Tối ưu hóa thứ tự nước đi	7
6 Bảng hoán vị và Zobrist Hashing	8
6.1 Zobrist Hashing	8
6.2 Cấu trúc TT_Entry và chiến lược ghi đè	8
6.3 Quy tắc sử dụng khi tra cứu	8
7 Quiescence Search	9
7.1 Hiệu ứng chân trời (Horizon Effect)	9
7.2 Giải pháp: Quiescence Search	9

8 Iterative Deepening và Aspiration Windows	10
8.1 Iterative Deepening Search (IDS)	10
8.2 Aspiration Windows	11
9 Hàm đánh giá	11
9.1 Vật chất ($w_m = 1,0$)	12
9.2 Vị trí - Piece-Square Table ($w_p = 1,2$)	12
9.3 An toàn Tượng ($w_k = 0,5$)	12
9.4 Cơ động ($w_b = 0,1$)	13
9.5 Xe mở cột/hàng ($w_r = 0,6$)	13
9.6 Pha trò chơi (Game Phase)	13
10 Sổ khai cuộc (Opening Book)	14
11 Tổng hợp tính năng và kiến trúc	15
12 Thảo luận và hướng phát triển	15
12.1 Nhận xét thiết kế	15
12.2 Hạn chế hiện tại	16
12.3 Hướng phát triển	16
13 Kết luận	16

1 Giới thiệu

Cờ Tướng (Xiangqi, 象棋) là một trong những trò chơi bàn cờ phổ biến nhất thế giới. So với cờ Vua, cờ Tướng có những đặc trưng riêng: bàn cờ 9×10 ô với sông ngăn đôi, luật cung cấm (Tướng và Sĩ không ra khỏi cung), Pháo bắt ăn bằng cách nhảy qua đúng một quân chắn, và luật Chiếu đối mặt. Không gian trạng thái ước tính lên tới 10^{150} thế cờ, đặt ra thách thức lớn cho mọi thuật toán tìm kiếm.

Phạm vi báo cáo

Báo cáo tập trung vào **các thuật toán và kỹ thuật AI** của engine. Chi tiết cấu trúc thư mục, API, và hướng dẫn cài đặt được ghi trong `README.md` của repository. Các phần chính gồm:

1. Biểu diễn bàn cờ và sinh nước đi hợp lệ (§2)
2. Thuật toán tìm kiếm Negamax - Alpha-Beta (§3)
3. Kỹ thuật cắt tỉa Null Move (§4)
4. Tối ưu hóa thứ tự nước đi (§5)
5. Bảng hoán vị và Zobrist Hashing (§6)
6. Quiescence Search (§7)
7. Iterative Deepening và Aspiration Windows (§8)
8. Hàm đánh giá tuyến tính (§9)
9. Sổ khai cuộc Opening Books (§10)

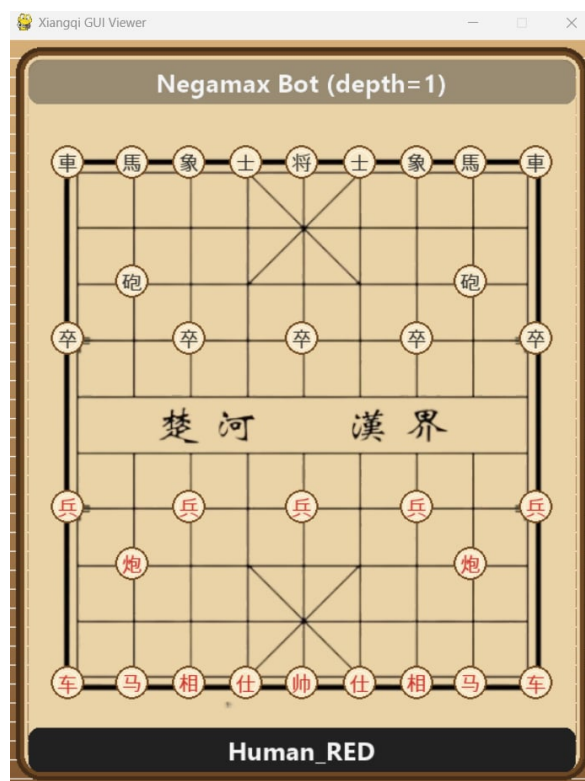
2 Biểu diễn bàn cờ và sinh nước đi

2.1 Mô hình bàn cờ

Bàn cờ được biểu diễn dưới dạng mảng tuyến tính state (với chế độ CLI) gồm $9 \times 10 = 90$ phần tử (`core/board.py`). Ô thứ i ứng với hàng $\lfloor i/9 \rfloor$ và cột $i \bmod 9$ (hàng 0 là sân Đen, hàng 9 là sân Đỏ). Ô trống ký hiệu là '+' (Hình 1). Hoặc sẽ được biểu diễn như một bàn cờ truyền thống trong chế độ GUI (Hình 2).

	a	b	c	d	e	f	g	h	i	
0	車	馬	象	士	將	士	象	馬	車	0
1	+	+	+	+	+	+	+	+	+	1
2	+	砲	+	+	+	+	+	砲	+	2
3	卒	+	卒	+	卒	+	卒	+	卒	3
4	+	+	+	+	+	+	+	+	+	4
5	+	+	+	+	+	+	+	+	+	5
6	兵	+	兵	+	兵	+	兵	+	兵	6
7	+	炮	+	+	+	+	+	炮	+	7
8	+	+	+	+	+	+	+	+	+	8
9	車	馬	相	仕	帥	仕	相	馬	車	9
	a	b	c	d	e	f	g	h	i	

Hình 1: Bàn cờ trong chế độ CLI



Hình 2: Bàn cờ trong chế độ GUI

2.2 Mã hóa quân cờ

Quân cờ được mã hóa bằng ký tự ASCII đơn: chữ hoa cho phe Đỏ, chữ thường cho phe Đen.

Bảng 1: Mã hóa quân cờ và giá trị vật chất (đơn vị: centipawn).

Quân	Đỏ/Đen	Hán tự	Giá trị	Đặc điểm di chuyển
Tướng	K/k	帅/将	6 000	1 bước ngang/dọc trong cung; không nhìn thẳng nhau.
Sĩ	A/a	仕/士	120	1 bước chéo trong cung.
Tượng	E/e	相/象	120	2 bước chéo, không qua sông; bị chặn nếu “chân voi” bị cản.
Mã	H/h	马/馬	270	1 dọc/ngang + 1 chéo; bị chặn nếu bước thẳng bị cản.
Xe	R/r	车/車	600	Bất kỳ số bước ngang/dọc không bị cản.
Pháo	C/c	炮/砲	285	Di chuyển như Xe; ăn quân phải nhảy qua đúng 1 quân chắn.
Tốt	P/p	兵/卒	30	1 bước tiến trước khi qua sông; tiến hoặc ngang sau khi qua.

2.3 Sinh nước đi hợp lệ

Quá trình sinh nước đi thực hiện qua hai giai đoạn:

1. **Sinh giả-hợp-lệ (pseudo-legal):** Sinh tất cả nước đi tuân theo động học quân cờ, ràng buộc sông/cung, và quy tắc chặn (chân Mã, chân Tượng, quân chắn Pháo) – module `core/move_generator.py`.
2. **Lọc hợp lệ (legality filter):** Loại bỏ các nước đi để lại Tướng trong tình trạng bị chiếu, bao gồm luật Chiếu đối mặt (flying general) – module `core/rules.py`.

Mỗi nước đi được mã hóa thành int 32-bit bằng cách đóng gói ô xuất phát ([0, 89]) và ô đích ([0, 89]) qua bit-shift, cho phép so sánh và lưu trữ hiệu quả.

3 Thuật toán tìm kiếm

3.1 Từ Minimax đến Negamax

Cây trò chơi được mô hình hóa theo bài toán đối kháng hai người chơi tổng điểm bằng không (zero-sum), với các lượt MAX và MIN luân phiên. Thay vì duy trì hai hàm riêng, engine sử dụng dạng **Negamax**:

đảo dấu điểm số khi chuyển qua mỗi tầng, biểu diễn gọn trong một hàm đệ quy thống nhất:

Nguyên lý Negamax

$$\text{negamax}(s) = \max_{m \in M(s)} (-\text{negamax}(\text{apply}(s, m)))$$

Tại mọi node, điểm số của bên hiện tại bằng phủ định điểm số đối thủ. $M(s)$ là tập nước đi hợp lệ tại trạng thái s .

3.2 Cắt tỉa Alpha-Beta

Alpha-Beta pruning mở rộng Negamax bằng cách duy trì hai tham số:

- α : điểm tốt nhất đảm bảo cho bên MAX tính đến thời điểm hiện tại.
- β : điểm tốt nhất đảm bảo cho bên MIN tính đến thời điểm hiện tại.

Khi $\alpha \geq \beta$ (cutoff), toàn bộ cây con còn lại bị cắt mà không ảnh hưởng đến kết quả. Với thứ tự nước đi hoàn hảo, độ phức tạp giảm từ $O(b^d)$ xuống $O(b^{d/2})$, tương đương tăng gấp đôi độ sâu tìm kiếm trong cùng ngân sách thời gian.

3.3 Điểm số kết thúc ván

- **Nếu đỏ hoặc đen thắng:** return $-90000 + \text{depth}$.
- **Hòa:** 0.

4 Cắt tỉa Null Move

Ý tưởng: Nếu bên đi bỏ qua lượt (null move) mà điểm tìm kiếm vẫn $\geq \beta$, thì vị thế đã quá tốt - đối thủ sẽ tránh vào đây từ trước. Toàn bộ nhánh bị cắt mà không cần duyệt tiếp.

Cài đặt trong `algorithm.py`:

- Null move chỉ đổi `side_to_move` và XOR khóa Zobrist với `ZOBRIST_SIDE`; snapshot được lưu vào `board.history` để undo hoàn toàn.
- Độ sâu giảm: $d' = d - 1 - R$ với $R = 2$ (`_NULL_MOVE_R`).
- Tìm kiếm với cửa sổ null $[-\beta, -\beta + 1]$ (zero-window search).
- Nếu `null_score` $\geq \beta$: lưu `LOWERBOUND` vào TT và trả về β ngay.

Điều kiện kích hoạt - cả bốn phải thỏa mãn:

1. Độ sâu $d \geq 3$ (`_NULL_MOVE_MIN_DEPTH = 3`).
2. Nước trước không phải null move (tránh null-null liên tiếp, `is_null_move = False`).
3. Bên đi không đang bị chiếu (`is_in_check = False`).
4. Tổng số Xe + Mã + Pháo còn lại ≥ 2 (hàm `_is_null_move_ok`).
Điều kiện này tránh sai lầm zugzwang ở tàn cuộc cận quân.

Algorithm 1 Null Move Pruning

```

1: nếu  $d \geq 3$  và  $\neg isNullMove$  và  $\neg inCheck$  và NullOk(board) thì
2:   DoNullMove(board)
3:    $nullScore \leftarrow -\text{Negamax}(board, d-1-2, -\beta, -\beta+1, isNull=True)$ 
4:   UndoNullMove(board)
5:   nếu  $nullScore \geq \beta$  thì
6:     Store(..., LOWERBOUND); trả về  $\beta$ 
7:   kết thúc nếu
8: kết thúc nếu
  
```

5 Tối ưu hóa thứ tự nước đi

Với thứ tự nước đi hoàn hảo, Alpha-Beta đạt độ phức tạp $O(b^{d/2})$ thay vì $O(b^d)$. Engine cài đặt pipeline sắp xếp bốn tầng trong `bots/engine/move_ordering.py`:

Bảng 2: Pipeline sắp xếp nước đi bốn tầng (ưu tiên giảm dần).

Mức	Tầng	Mô tả và điểm sắp xếp
4	Hash Move	Nước đi lưu trong TT từ vòng IDS trước. Điểm: (4, 0).
3	MVV-LVA	Nước ăn quân: $score = \text{PIECE_VALUE}[\text{victim}] \times 1000 - \text{PIECE_VALUE}[\text{attacker}]$. Điểm: (3, MVV-LVA).
2	Killer 0	Nước non-capture gây β -cutoff gần nhất ở cùng ply. Điểm: (2, 0).
1	Killer 1	Nước non-capture gây β -cutoff thứ hai gần nhất. Điểm: (1, 0).
0	History	Điểm tích lũy từ bảng 90×90 : cộng $depth^2$ mỗi khi gây cutoff. Điểm: (0, <code>history[from][to]</code>).

Hàm sắp xếp trả về tuple (mc, im) và sort theo thứ tự giảm dần, đảm bảo Hash Move luôn đứng đầu, sau đó captures theo MVV-LVA, rồi killers, rồi các nước lạng lẽ theo history.

Killer Move Heuristic: Mỗi ply lưu hai khe killer (`_killers[depth][0:2]`). Khi một nước non-capture gây β -cutoff, nó đẩy vào khe 0, khe cũ dịch sang khe 1. Ở lần gặp lại cùng ply (node anh chị em), killer được ưu tiên cao hơn nước thường.

History Heuristic: Bảng `history[from][to]` kích thước 90×90 tích lũy $depth^2$ mỗi khi nước đi gây cutoff. Bảng này tồn tại xuyên suốt toàn bộ phiên tìm kiếm, phản ánh các mẫu chiến thuật tái xuất hiện ở nhiều nhánh.

6 Bảng hoán vị và Zobrist Hashing

6.1 Zobrist Hashing

Mỗi thế cờ được định danh bằng khóa Zobrist 64-bit. Tại khởi tạo, sinh ma trận số ngẫu nhiên $Z_{p,s}$ cho mọi cặp (quân cờ p , ô $s \in [0, 89]$) và một khóa riêng Z_{side} cho lượt đi.

Khóa được **cập nhật gia tăng** (incremental update) khi thực hiện/hoàn tác nước đi - tránh quét lại toàn bàn cờ:

$$\mathcal{H}_{\text{mới}} = \mathcal{H}_{\text{cũ}} \oplus Z_{p, \text{from}} \oplus Z_{p, \text{to}} \oplus Z_{\text{side}} \oplus \begin{cases} Z_{\pi(\text{to}), \text{to}} & \text{nếu ô đích có quân } \pi(\text{to}) \\ 0 & \text{nếu ô đích trống} \end{cases}$$

6.2 Cấu trúc TT_Entry và chiến lược ghi đè

Bảng 3: Cấu trúc một ô trong Bảng hoán vị (TT_Entry).

Trường	Kiểu	Ý nghĩa
key	int 64-bit	Khóa Zobrist để xác nhận trùng cache (phòng collision).
depth	int	Độ sâu tìm kiếm khi lưu entry.
score	float	Điểm minimax tính được.
flag	TT_FLAG	EXACT (0), LOWERBOUND (1), UPPERBOUND (2).
best_move	int	Nước đi tốt nhất - dùng cho Hash Move ở vòng sau.

Chiến lược ghi đè: Entry mới ghi đè khi ô trống hoặc $depth_{\text{mới}} \geq depth_{\text{cũ}}$ - ưu tiên giữ thông tin tìm kiếm sâu hơn.

6.3 Quy tắc sử dụng khi tra cứu

Khi probe trả về entry hữu dụng ($depth_e \geq d$):

- **EXACT:** Trả về $score_e$ ngay.

- **LOWERBOUND:** $\alpha \leftarrow \max(\alpha, score_e)$.
- **UPPERBOUND:** $\beta \leftarrow \min(\beta, score_e)$.
- Nếu $\alpha \geq \beta$ sau điều chỉnh: cutoff, trả về $score_e$.

Xác định flag khi lưu:

- $maxScore \leq origAlpha$: UPPERBOUND (không cải thiện α).
- $maxScore \geq \beta$: LOWERBOUND (gây β -cutoff).
- Còn lại: EXACT.

7 Quiescence Search

7.1 Hiệu ứng chân trời (Horizon Effect)

Thuật toán tìm kiếm độ sâu cố định d chịu *horizon effect*: các sự kiện chiến thuật xảy ra ở tầng $d + 1$ (ăn quân lớn, chiếu không chặn được) bị hàm đánh giá tính bỏ qua – engine có thể chọn nước đi tệ chỉ vì thế cờ “trông ổn” tại tầm nhìn cận.

7.2 Giải pháp: Quiescence Search

Khi tìm kiếm chính đạt $depth = 0$, thay vì gọi hàm đánh giá ngay, engine chuyển sang **Quiescence Search**: tiếp tục tìm kiếm chỉ xét các *nước ăn quân* (captures) cho đến khi đạt trạng thái yên tĩnh (không còn capture khả dụng) hoặc đạt giới hạn $Q_{max} = 4$.

Stand-pat: Tại mỗi node, tính đánh giá tĩnh $stand_pat$. Nếu $stand_pat \geq \beta$: β -cutoff ngay. Nếu $stand_pat > \alpha$: cập nhật α (bên hiện tại có thể chọn “không ăn quân” và giữ điểm này).

Algorithm 2 Quiescence Search

```

1: Hàm QuiescenceSearch(board,  $\alpha$ ,  $\beta$ , qdepth)
2:   standPat  $\leftarrow$  Heuristic(board)
3:   nếu standPat  $\geq \beta$  thì trả về  $\beta$ 
4:   kết thúc nếu
5:   nếu standPat  $> \alpha$  thì  $\alpha \leftarrow \text{standPat}$ 
6:   kết thúc nếu
7:   nếu qdepth  $\geq 4$  thì trả về  $\alpha$ 
8:   kết thúc nếu
9:   captures  $\leftarrow [m \in \text{GetLegalMoves}(\text{board}) \mid \text{IsCapture}(\text{board}, m)]$ 
10:  nếu captures rỗng thì trả về  $\alpha$ 
11:  kết thúc nếu
12:  captures  $\leftarrow \text{SortMoves}(\text{captures}, \text{depth}=0)$ 
13:  với mỗi move trong captures thực hiện
14:    board.MakeMove(move)
15:    score  $\leftarrow -\text{QuiescenceSearch}(\text{board}, -\beta, -\alpha, \text{qdepth}+1)$ 
16:    board.UndoMove
17:    nếu score  $\geq \beta$  thì trả về  $\beta$ 
18:    kết thúc nếu
19:    nếu score  $> \alpha$  thì  $\alpha \leftarrow \text{score}$ 
20:    kết thúc nếu
21:  kết thúc với mỗi
22:  trả về  $\alpha$ 
23: kết thúc Hàm

```

Lưu ý: engine dùng `get_legal_moves` (fully legal) thay vì pseudo-legal để đảm bảo không tạo trạng thái thiếu Tướng, tránh crash hàm `_find_king()` ở bước đánh giá an toàn Tướng.

8 Iterative Deepening và Aspiration Windows

8.1 Iterative Deepening Search (IDS)

Engine không tìm kiếm trực tiếp đến độ sâu cố định mà dùng chiến lược **Iterative Deepening**: lần lượt tìm kiếm $d = 1, 2, 3, \dots$ đến khi hết ngân sách thời gian. Engine cung cấp hai chế độ:

- `search_with_time_limit`: dừng khi đã dùng $> 90\%$ thời gian, đảm bảo luôn có *best move so far* để trả về nếu bị ngắt.
- `search_with_depth_limit`: tìm đến `max_depth` cố định (chủ yếu dùng cho kiểm thử).

Lý do IDS không lãng phí: Số node của vòng lặp d chiếm xấp xỉ $b/(b-1)$ tổng số node của tất cả vòng trước cộng lại, nên chi phí lặp lại

vòng nông là không đáng kể. Đổi lại, Hash Move và History Heuristic từ vòng $d - 1$ cải thiện đáng kể thứ tự nước đi cho vòng d .

8.2 Aspiration Windows

Ý tưởng: Từ độ sâu $d \geq 4$ (`_ASPIRATION_MIN_DEPTH`), thay vì dùng cửa sổ mở $[-\infty, +\infty]$, engine khởi tạo cửa sổ hẹp $[\hat{s} - \Delta, \hat{s} + \Delta]$ với $\Delta = 50$ (`_ASPIRATION_DELTA`) quanh điểm số $\hat{s}$ của vòng lặp trước. Cửa sổ hẹp tăng tần suất cutoff, giảm số node cần duyệt.

Xử lý fail:

- **Fail-low** ($score \leq \hat{s} - \Delta$): mở rộng cửa sổ xuống $[-\infty, score + 1]$, tìm kiếm lại.
- **Fail-high** ($score \geq \hat{s} + \Delta$): mở rộng cửa sổ lên $[score - 1, +\infty]$, tìm kiếm lại.

Vòng lặp aspiration tiếp tục cho đến khi kết quả nằm trong cửa sổ. Trong trường hợp bình thường (không fail), tiết kiệm thời gian đáng kể; trong trường hợp fail, chi phí tìm kiếm lại chỉ xảy ra ở độ sâu nông và chấp nhận được.

Algorithm 3 Iterative Deepening với Aspiration Windows

```

1: với mỗi  $d \leftarrow 1$  đến  $maxDepth$  thực hiện
2:   nếu  $d \geq 4$  thì  $\alpha \leftarrow \hat{s} - 50$ ;  $\beta \leftarrow \hat{s} + 50$ 
3:   ngược lại  $\alpha \leftarrow -\infty$ ;  $\beta \leftarrow +\infty$ 
4:   kết thúc nếu
5:    $done \leftarrow \text{False}$ 
6:   trong khi  $\neg done$  thực hiện
7:      $(bestScore, bestMove) \leftarrow \text{SearchAllRootMoves}(board, d, \alpha, \beta)$ 
8:     nếu  $bestScore \leq \hat{s} - 50$  thì
9:        $\alpha \leftarrow -\infty$ ;  $\beta \leftarrow bestScore + 1$  ▷ Fail-low: mở rộng xuống
10:    ngược lại nếu  $bestScore \geq \hat{s} + 50$  thì
11:       $\alpha \leftarrow bestScore - 1$ ;  $\beta \leftarrow +\infty$  ▷ Fail-high: mở rộng lên
12:    ngược lại  $done \leftarrow \text{True}$ 
13:    kết thúc nếu
14:    kết thúc trong khi
15:     $\hat{s} \leftarrow bestScore$ ; cập nhật  $bestMove$ 
16: kết thúc với mỗi
17: trả về  $bestMove$ 

```

9 Hàm đánh giá

Hàm đánh giá $E(s)$ ước tính lợi thế tĩnh của phe Đỏ tại thế cờ s . Hàm heuristic trả về điểm từ góc nhìn bên đang đi (*side-relative*): nếu đang là Đỏ thì trả về E , Đen thì $-E$.

$$E = w_m \cdot E_{\text{vật chất}} + w_p \cdot E_{\text{vị trí}} + w_k \cdot E_{\text{an toàn}} + w_b \cdot E_{\text{cơ động}} + w_r \cdot E_{\text{Xe mở}} \quad (1)$$

9.1 Vật chất ($w_m = 1,0$)

Tổng giá trị quân Đỏ trừ tổng giá trị quân Đen theo bảng PIECE_VALUES trong `core/pieces.py` (xem Bảng 1).

9.2 Vị trí - Piece-Square Table ($w_p = 1,2$)

Mỗi loại quân có một PST 9×10 mã hóa kiến thức chuyên gia:

- **Xe (R):** Thưởng hàng 7 (vị trí tấn công chuẩn); thưởng cột trung tâm phía địch; phạt nặng khi vào cột d-e-f ở hàng 8-9 (gần cung mình).
- **Mã (H):** Thưởng vị trí triển khai hàng 7 (c7, g7); thưởng tiến sang sông; phạt góc hàng 9.
- **Pháo (C):** Thưởng khai cuộc b7/h7; thưởng tiến lên hàng 4-5 để tấn công; phạt khi lùi về góc a9/i9.
- **Tốt (P):** Điểm 0 trước khi qua sông; thưởng mạnh sau khi qua, đặc biệt cột trung tâm d-e-f.
- **Sĩ (A), Tượng (E):** Thưởng các ô chuẩn trong/bán cung.
- **Tướng (K):** Thưởng nhỏ vị trí giữa cung (hàng 7-9 cột d-f).

Phe Đỏ dùng PST trực tiếp (hàng 9 = sân Đỏ); phe Đen dùng PST lật ô: PST[89 - sq].

Trọng số $w_p = 1,2 > w_m = 1,0$ phản ánh triết lý: ưu tiên triển khai quân tốt hơn lợi thế vật chất nhỏ.

9.3 An toàn Tướng ($w_k = 0,5$)

Nguy hiểm của Tướng mỗi phe được tính từ hai thành phần:

1. **Shield penalty:** $(4 - \text{số Sĩ} + \text{Tượng còn lại}) \times 20$. Mỗi quân phòng thủ bị mất tăng nguy cơ cho Tướng.
2. **Tropism:** Tổng ảnh hưởng các quân địch theo khoảng cách Chebyshev đến Tướng:

$$\text{tropism} = \sum_{p_{\text{địch}}} \frac{w(p) \times 10}{\max(\text{Chebyshev}(p, \text{Tng}), 1)}$$

với trọng số: Xe/xe = 4, Pháo/pháo = 3, Mã/mã = 3, Tốt/tốt = 1.

Tropism được điều chỉnh theo pha trò chơi ϕ : $\text{tropism} \leftarrow \text{tropism} \times (0,4 + 0,6 \times \phi)$ – quan trọng hơn ở khai cuộc, giảm dần về tàn cuộc.

$$E_{\text{an toàn}} = \text{danger}_{\text{Đen}} - \text{danger}_{\text{Đỏ}}$$

Trọng số $w_k = 0,5$ được giảm so với mặc định để bot bớt thụ động, tránh lùi quân về phòng thủ quá mức.

9.4 Cơ động ($w_b = 0,1$)

Đếm số nước giả-hợp-lệ (pseudo-legal) của Đỏ trừ Đen. Trọng số nhỏ vì đếm pseudo-legal dễ bị nhiễu ở vị trí đông quân.

9.5 Xe mở cột/hàng ($w_r = 0,6$)

Xe chiếm cột mở (không có quân cùng phe trên cùng cột) nhận bonus +15; hàng mở nhận +10. Lợi thế chiến lược quan trọng ở trung cuộc.

9.6 Pha trò chơi (Game Phase)

$$\phi = \frac{\sum_{p \notin \{K,k\}} \text{PIECE_VALUE}(p) - 1200}{2400 - 1200}, \quad \phi \in [0, 1]$$

$\phi = 1,0$: khai cuộc (vật chất ≥ 2400); $\phi = 0,0$: tàn cuộc (vật chất ≤ 1200); pha giữa nội suy tuyến tính.

Bảng 4: Tổng hợp các thành phần hàm đánh giá.

Thành phần	Trọng số	Ghi chú
Vật chất	1,0	Điểm neo; các thành phần khác tỉ lệ tương đối.
Vị trí (PST)	1,2	Cao nhất – engine ưu tiên triển khai quân tốt.
An toàn Tướng	0,5	Giảm để bot bớt thụ động.
Cơ động	0,1	Pseudo-legal; trọng số thấp để tránh nhiễu.
Xe mở cột/hàng	0,6	Lợi thế chiến lược trung cuộc.

10 Sổ khai cuộc (Opening Book)

Engine tích hợp sổ khai cuộc trong `bots/engine/opening_book.py` dưới dạng từ điển:

`OPENING_BOOK: int → list[int]`

với **khóa** là giá trị Zobrist của thế cờ và **giá trị** là danh sách nước đi khuyến dùng (mã hóa int).

Cơ chế tra cứu: Trước khi gọi IDS, engine kiểm tra `board.zobrist_key` trong `OPENING_BOOK`. Nếu trùng, chọn ngẫu nhiên một nước từ danh sách - tạo sự đa dạng trong khai cuộc mà không tốn thời gian tìm kiếm.

Sổ bao gồm tám hệ thống khai cuộc chuẩn:

Bảng 5: Các hệ thống khai cuộc trong sổ.

Tên khai cuộc	Mô tả
Pháo Đầu (Central Cannon)	Pháo h7e7 - phổ biến nhất; đối phó với Bình Phong Mã, Thuận Pháo, Nghịch Pháo.
Tiên Nhân Chỉ Lộ (Pawn)	Tiến Tốt g6g5 - linh hoạt, dễ kết hợp với nhiều hệ thống.
Khởi Mã (Edge Horse)	Mã h9g7 - triển khai Mã nhanh, kiểm soát cột trung.
Phi Tượng (Elephant Opening)	Tượng g9e7 - phòng thủ vững, ít tính tấn công.
Thuận Pháo	Cả hai Pháo cùng chiều - thế cờ cân bằng, nhiều biến thể.
Nghịch Pháo	Hai Pháo ngược chiều - phức tạp, nhiều chiến thuật.
Sĩ Giác Pháo	Pháo h7f7 - biến thể tấn công hông, bất ngờ.
Quá Cung Pháo	Pháo h7d7 - đe dọa từ xa, ít phổ biến.

11 Tổng hợp tính năng và kiến trúc

Bảng 6: Tổng hợp toàn bộ tính năng của engine.

Hạng mục	Tính năng	Tham số / Ghi chú
Tìm kiếm cốt lõi	Negamax + Alpha-Beta	Khung thống nhất, side-relative score
	Quiescence Search	$Q_{\max} = 4$; chỉ xét captures
Cắt tỉa nâng cao	Null Move Pruning	$R = 2$; min_depth = 3; bảo vệ zugzwang
Thứ tự nước đi	Hash Move	Tầng 4 - ưu tiên cao nhất
	MVV-LVA Captures	Tầng 3 - victim $\times 1000$ - attacker
	Killer Move Heuristic History Heuristic	Tầng 2 & 1 - 2 khe mỗi ply Tầng 0 - bảng 90×90 , cộng $depth^2$
Bộ nhớ đệm	Transposition Table	Zobrist 64-bit; EXACT/LOWER/UPPER
	Zobrist Hashing	Cập nhật gia tăng; ghi đè khi depth sâu hơn
Điều hướng thời gian	Iterative Deepening	Time-limit (90%) và depth-limit mode
	Aspiration Windows	$\Delta = 50$; từ depth ≥ 4 ; fail xử lý mở rộng
Hàm đánh giá	Vật chất ($w = 1,0$)	Theo PIECE_VALUES
	PST ($w = 1,2$)	7 bảng 9×10 ; Đen dùng PST[89-sq]
	An toàn Tướng ($w = 0,5$)	Shield + Tropism Chebyshev
	Cơ động ($w = 0,1$)	Pseudo-legal; trọng số thấp
	Xe mở cột/hàng ($w = 0,6$)	Bonus +15 (cột) / +10 (hàng)
Khai cuộc	Opening Book	Zobrist lookup; 8 hệ thống
Luật cờ	7 loại quân	Chiếu đối mặt; phát hiện hòa

12 Thảo luận và hướng phát triển

12.1 Nhận xét thiết kế

Các kỹ thuật trong engine tạo thành vòng tự tăng cường: sắp xếp nước đi tốt giúp Alpha-Beta cắt nhiều hơn; TT cung cấp Hash Move cho sắp xếp; IDS tích lũy History Heuristic xuyên vòng lặp; Aspiration Windows tận dụng điểm số từ vòng trước; Null Move giảm node ở phần

lớn nhánh không tiềm năng.

Trọng số $w_p = 1,2 > w_m = 1,0$ phản ánh triết lý thiết kế: ưu tiên vị trí hơn lợi thế vật chất nhỏ – phù hợp với phong cách chơi chủ động của Cờ Tướng. Trọng số $w_k = 0,5$ giảm để bot không thụ động phòng thủ quá mức.

12.2 Hạn chế hiện tại

- Trọng số hàm đánh giá đặt thủ công, chưa tối ưu hóa tự động (SPSA, self-play tuning).
- Số khai cuộc còn hạn chế về chiều sâu và số lượng biến thể.
- Engine đơn luồng, chưa tận dụng CPU đa nhân.
- Chưa có bảng PST riêng cho tàn cuộc (tapered evaluation đầy đủ với hai bộ PST opening/endgame nội suy theo ϕ).
- Chưa có endgame tablebases cho tàn cuộc cận quân.

12.3 Hướng phát triển

1. **Monte Carlo Tree Search (MCTS):** Kết hợp hoặc thay thế Alpha-Beta để cải thiện tầm nhìn dài hạn, bớt phụ thuộc vào hàm đánh giá thủ công.
2. **Mạng nơ-ron đánh giá:** Huấn luyện CNN/ResNet trên dữ liệu tự đấu theo hướng AlphaZero để thay thế hàm đánh giá.
3. **Parallel search:** Lazy SMP hoặc Principal Variation Splitting để khai thác CPU đa nhân.
4. **Mở rộng số khai cuộc:** Tích hợp cơ sở dữ liệu khai cuộc lớn hơn từ các ván đấu chuyên nghiệp.
5. **Endgame tablebases:** Tra cứu kết quả chính xác ở tàn cuộc cận quân.

13 Kết luận

Báo cáo này trình bày một engine Cờ Tướng tích hợp các kỹ thuật: Negamax Alpha-Beta với Null Move Pruning; Iterative Deepening kết hợp Aspiration Windows; Quiescence Search; pipeline sắp xếp nước đi bốn tầng (Hash Move, MVV-LVA, Killer, History); Transposition Table với Zobrist Hashing; hàm đánh giá tuyến tính năm thành phần có nội suy pha trò chơi; và Số khai cuộc Zobrist với tám hệ thống chuẩn.

Codebase được cài đặt bằng Python: lớp luật cờ (core/), lớp thuật toán (bots/engine/), và lớp điều phối (bots/bot.py).

Toàn bộ mã nguồn công khai tại github.com/Oxigo-1212/INT3401E-2-project.